



03. Nested Documents & Deletion Operations

Nested Documents & Dot Notation

MongoDB documents can embed child objects or arrays. Query nested fields using **Dot Notation** (always wrapped in quotes!):

```
mongosh — Nested Query

// Document structure: { name: "Aman", performance: { marks: 88, grade: "A" } }
db.student.find({ "performance.marks": { $gte: 85 } });
// Quotes ( " ") are strictly mandatory when using dot notation in queries!
```



Deletion Operations

Command	Purpose	Return Value
db.student.deleteOne(filter)	Deletes first matched document.	{ acknowledged: true, deletedCount: 1 }
db.student.deleteMany(filter)	Deletes all matched documents.	{ acknowledged: true, deletedCount: N }
db.student.drop()	Drops the entire 'student' collection.	true
db.dropDatabase()	Permanently drops the currently active database.	{ ok: 1, dropped: "college" }

```
mongosh — Deletions

// 1. Delete single document
db.student.deleteOne({ name: "Amit Patel" });

// 2. Delete all students with marks less than 40
db.student.deleteMany({ marks: { $lt: 40 } });
```



CAUTION: Executing `deleteMany({})` with an empty filter will delete all documents in the collection permanently!



Introduction to Mongoose ODM & Connection



What is Mongoose ODM?

Mongoose is an **Object Data Modeling (ODM)** library for Node.js. It acts as an abstraction bridge between JavaScript application objects and MongoDB documents.

Node.js App
(JS Objects)

⇔ Mongoose ODM ⇔
(Schema, Validations, Casting)

MongoDB Server
(BSON Documents)



Connection String Anatomy

mongodb://127.0.0.1:27017/collegeDB

Protocol | Host IP (localhost) | Default Port | Database Name



Connecting: Promises vs Async/Await



index.js — Mongoose Connection

```
const mongoose = require("mongoose");

// Modern Async/Await Pattern (Industry Standard)
async function main() {
  await mongoose.connect("mongodb://127.0.0.1:27017/collegeDB");
}
main()
  .then(() => console.log("✅ MongoDB Connected Successfully!"))
  .catch((err) => console.log("❌ Connection Error:", err));
```



WHY NOT NATIVE DRIVER? The native MongoDB driver lacks schema enforcement, risking bugs from invalid fields or type mismatches. Mongoose provides robust validation and structure!